

Tidy Data

Load and Inspect Data

```
In [1]: import os
import pandas as pd

base_dir = os.getcwd()
data_dir = os.path.join(base_dir, 'Ausgrid_solar_home_data')

csv_filepath_1 = os.path.join(data_dir, 'Solar home 2010-2011.csv')
csv_filepath_2 = os.path.join(data_dir, 'Solar home 2011-2012.csv')
csv_filepath_3 = os.path.join(data_dir, 'Solar home 2012-2013.csv')

# Load the three yearly files
df_2010 = pd.read_csv(csv_filepath_1, skiprows=1)
df_2011 = pd.read_csv(csv_filepath_2, skiprows=1)
df_2012 = pd.read_csv(csv_filepath_3, skiprows=1)

# Inspect each file
for name, df in [('2010-2011', df_2010), ('2011-2012', df_2011), ('2012-2013', df_2012)]:
    print(f"Name: {name}")
    print(f"Shape: {df.shape}")
    print(f"Columns:", df.columns.tolist())
    print(f"Missing values: {df.isna().sum().sum()}\n")
```

```

Name: 2010-2011
Shape: (269735, 53)
Columns: ['Customer', 'Generator Capacity', 'Postcode', 'Consumption Category', 'date', '0:30', '1:00', '1:30', '2:00', '2:30', '3:00', '3:30', '4:00', '4:30', '5:00', '5:30', '6:00', '6:30', '7:00', '7:30', '8:00', '8:30', '9:00', '9:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30', '13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30', '17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30', '21:00', '21:30', '22:00', '22:30', '23:00', '23:30', '0:00']
Missing values: 0

```

```

Name: 2011-2012
Shape: (270304, 54)
Columns: ['Customer', 'Generator Capacity', 'Postcode', 'Consumption Category', 'date', '0:30', '1:00', '1:30', '2:00', '2:30', '3:00', '3:30', '4:00', '4:30', '5:00', '5:30', '6:00', '6:30', '7:00', '7:30', '8:00', '8:30', '9:00', '9:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30', '13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30', '17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30', '21:00', '21:30', '22:00', '22:30', '23:00', '23:30', '0:00', 'Row Quality']
Missing values: 270304

```

```

Name: 2012-2013
Shape: (268557, 54)
Columns: ['Customer', 'Generator Capacity', 'Postcode', 'Consumption Category', 'date', '0:30', '1:00', '1:30', '2:00', '2:30', '3:00', '3:30', '4:00', '4:30', '5:00', '5:30', '6:00', '6:30', '7:00', '7:30', '8:00', '8:30', '9:00', '9:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30', '13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30', '17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30', '21:00', '21:30', '22:00', '22:30', '23:00', '23:30', '0:00', 'Row Quality']
Missing values: 268557

```

The documentation says `Row Quality` distinguishes actual (blank) readings from non-actual (NA) readings where some or all half-hour values were estimated or substituted. Let's check to see if the missing values are coming from the `Row Quality` column.

```

In [2]: print(df_2011["Row Quality"].value_counts(dropna=False))
        print(df_2012["Row Quality"].value_counts(dropna=False))

```

```

Row Quality
NaN      270304
Name: count, dtype: int64
Row Quality
NaN      268557
Name: count, dtype: int64

```

The only value is NaN, so we can drop the column and proceed.

Combine all three year tables

```

In [3]: df_2011 = df_2011.drop(columns=["Row Quality"], errors="ignore")
        df_2012 = df_2012.drop(columns=["Row Quality"], errors="ignore")
        df = pd.concat([df_2010, df_2011, df_2012], ignore_index=True)

```

```
# Inspect table
print(f"Name: Combined 2010-2013")
print(f"Shape: {df.shape}")
print(f"Columns:", df.columns.tolist())
print(f"Missing values: {df.isna().sum().sum()}\n")
```

Name: Combined 2010-2013

Shape: (808596, 53)

Columns: ['Customer', 'Generator Capacity', 'Postcode', 'Consumption Category', 'date', '0:30', '1:00', '1:30', '2:00', '2:30', '3:00', '3:30', '4:00', '4:30', '5:00', '5:30', '6:00', '6:30', '7:00', '7:30', '8:00', '8:30', '9:00', '9:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30', '13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30', '17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30', '21:00', '21:30', '22:00', '22:30', '23:00', '23:30', '0:00']

Missing values: 0

Reformat and Tidy Data

Convert wide format to tidy long format. Replace `date` and `time` columns with proper datetime column. The `date` column is inconsistently formatted, so we use

`format="mixed"` to handle multiple forms such as `1-Jul-10` and `1/07/2011`.

```
In [4]: # Half-hour columns
time_cols = df.columns[5:]

df_long = df.melt(
    id_vars=df.columns[:5],
    value_vars=time_cols,
    var_name='time',
    value_name='kWh')

# Create timestamp
df_long['datetime'] = pd.to_datetime(
    df_long['date'].astype(str) + " " + df_long["time"],
    format="mixed",
    dayfirst=True)

# Drop redundant date and time columns
df_long = df_long.drop(columns=["date", "time"])

# Optional: sort
df_long = df_long.sort_values(['Customer', 'Consumption Category', 'datetime']).reset_index()

# Optional: change column order
df_long = df_long[[
    "Customer", "Generator Capacity", "Postcode",
    "Consumption Category", "datetime", "kWh"]]

# Inspect table
display(df_long.head())
print(f"Shape: {df_long.shape}")
```

```
print(f"Columns:", df_long.columns.tolist())
print(f"Missing values: {df_long.isna().sum().sum()}\n")
```

	Customer	Generator Capacity	Postcode	Consumption Category	datetime	kWh
0	1	3.78	2076	CL	2010-07-01 00:00:00	1.075
1	1	3.78	2076	CL	2010-07-01 00:30:00	1.250
2	1	3.78	2076	CL	2010-07-01 01:00:00	1.244
3	1	3.78	2076	CL	2010-07-01 01:30:00	1.256
4	1	3.78	2076	CL	2010-07-01 02:00:00	0.744

Shape: (38812608, 6)

Columns: ['Customer', 'Generator Capacity', 'Postcode', 'Consumption Category', 'datetime', 'kWh']

Missing values: 0

Save Data

```
In [5]: csv_save_filepath = os.path.join(data_dir, 'ausgrid_processed.csv')
df_long.to_csv(csv_save_filepath, index=False)
```

Analyze Data

Customer-level solar statistics

kWp means kilowatt-peak: the rated maximum power capacity of the solar panels under standard test conditions. The units are different because kWp is power capacity, while kWh is energy produced over time. Dividing them gives an energy-per-unit-capacity measure, which is useful for comparing differently sized solar systems. The Ausgrid documentation itself reports annual gross generation in **kWh/kWp**.

$$\frac{\text{kWh}}{\text{kWp}} = \frac{\text{energy generated (kWh)}}{\text{rated capacity (kWp)}}$$

```
In [6]: # Customer-level solar statistics
solar_stats = (
    df_long[df_long['Consumption Category'] == 'GG']
    .groupby('Customer')
    .agg(
```

```

generator_capacity_kwp=('Generator Capacity', 'first'),
annual_pv_kwh=('kWh', 'sum'))
.reset_index())

# PV generation normalized by system capacity
solar_stats['pv_kwh_per_kwp'] = (
    solar_stats['annual_pv_kwh'] / solar_stats['generator_capacity_kwp'])

display(solar_stats.head().style.hide(axis="index"))
print(solar_stats.drop(columns="Customer").describe().drop(index="count"))

```

Customer	generator_capacity_kwp	annual_pv_kwh	pv_kwh_per_kwp
1	3.780000	14708.534000	3891.146561
2	1.620000	5995.505000	3700.929012
3	1.000000	4419.744000	4419.744000
4	1.000000	3802.059000	3802.059000
5	1.000000	3658.742000	3658.742000
	generator_capacity_kwp	annual_pv_kwh	pv_kwh_per_kwp
mean	1.683300	6382.756697	3779.737463
std	1.036846	4177.495487	497.604430
min	1.000000	1927.716000	1752.469091
25%	1.020000	3953.146500	3464.586750
50%	1.500000	5312.645000	3826.478810
75%	2.000000	7002.282500	4145.007850
max	9.990000	43091.895000	4870.287143

The mean system size is 1.68 kWp and the median is 1.50 kWp, matching Ausgrid's documentation almost exactly. The annual_pv_kwh is actually a 3-year total because we combined all three years before summing. The mean of 6382.8 kWh corresponds to about 2128 kWh/year, which is close to Ausgrid's reported annual means of 2119, 2083, and 2181 kWh. Likewise, 3779 kWh/kWp over 3 years is about 1260 kWh/kWp per year, close to the documented value (~1231-1297). There is substantial variation between customers: system size ranges from 1 to 9.99 kWp, and normalized annual generation varies considerably.

Daily and Seasonal Patterns

In [7]: `import matplotlib.pyplot as plt`

```

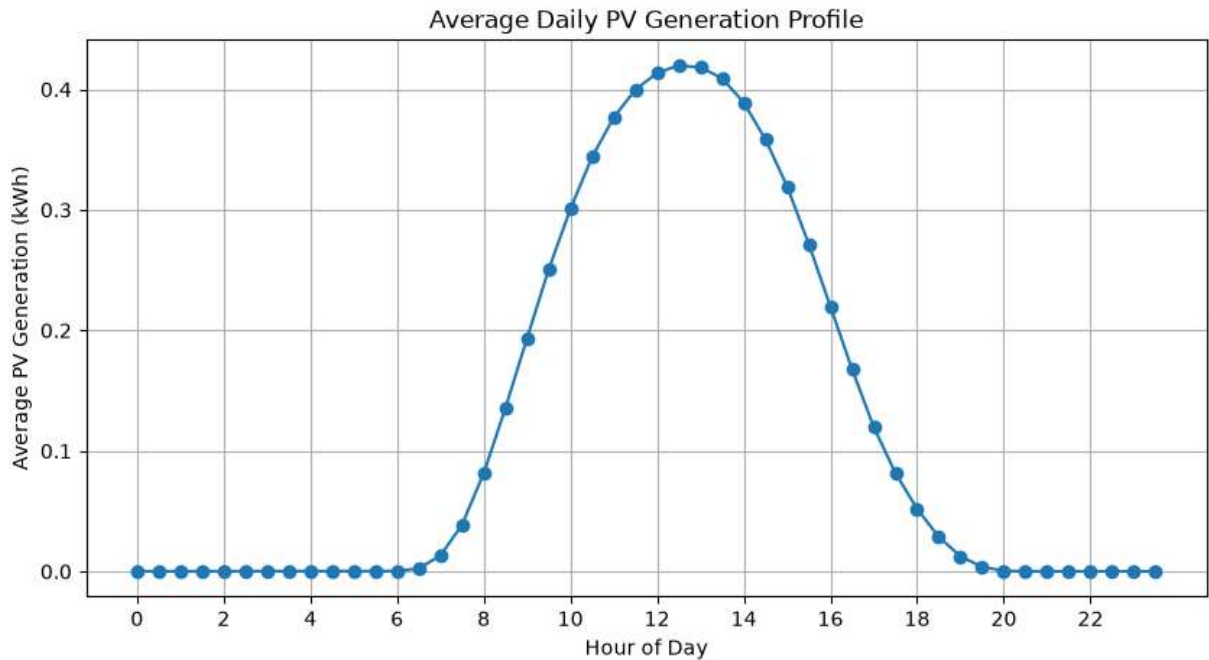
# PV data
pv = df_long[df_long["Consumption Category"] == "GG"].copy()

# Numeric hour of day
pv['hour'] = pv['datetime'].dt.hour + pv['datetime'].dt.minute / 60

# Daily pattern: average PV generation by half-hour
daily_profile = pv.groupby(pv['hour'])["kWh"].mean()

```

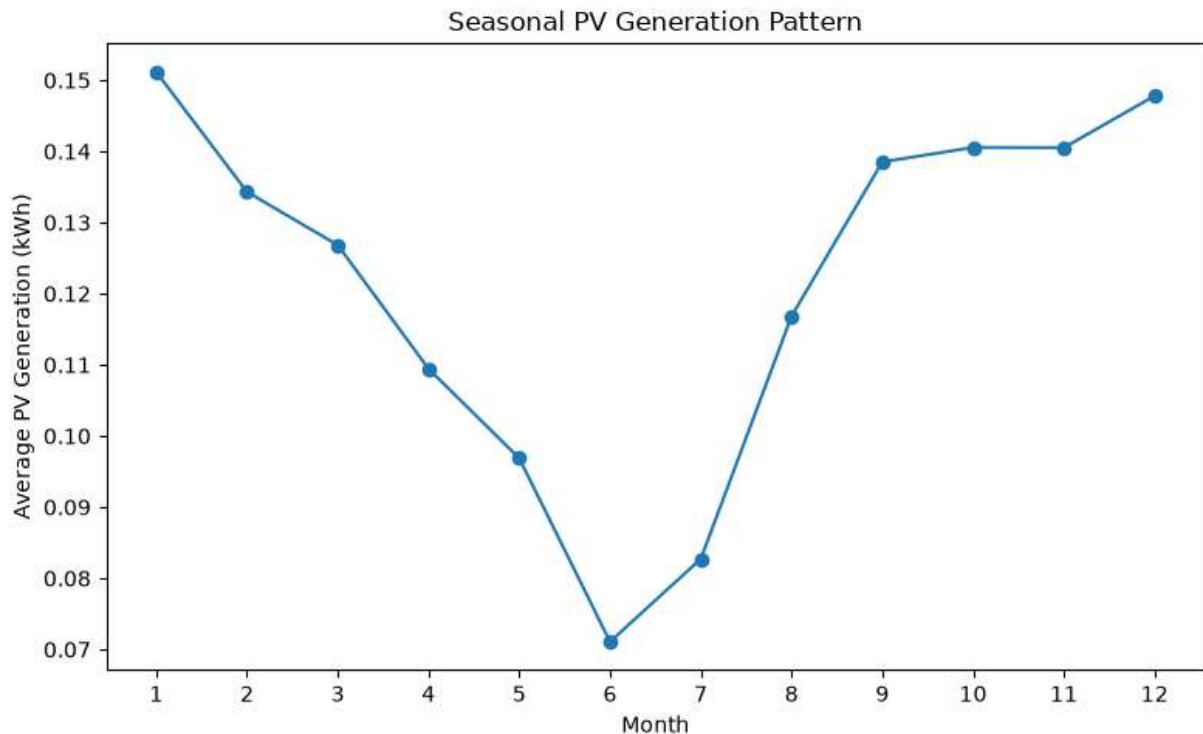
```
plt.figure(figsize=(10,5))
plt.plot(daily_profile.index, daily_profile.values, marker='o')
plt.xlabel("Hour of Day")
plt.ylabel("Average PV Generation (kWh)")
plt.title("Average Daily PV Generation Profile")
plt.xticks(range(0, 24, 2))
plt.grid()
plt.show()
```



The overall PV curve is essentially zero overnight, rises rapidly after about 7:00, peaks around 12:30-13:00, and falls back towards zero by about 19:00. Time of day is a very strong predictive feature.

```
In [8]: # Seasonal pattern: average PV generation by month
seasonal_profile = pv.groupby(pv['datetime'].dt.month)['kWh'].mean()

plt.figure(figsize=(8, 5))
plt.plot(seasonal_profile.index, seasonal_profile.values, marker="o")
plt.xlabel("Month")
plt.ylabel("Average PV Generation (kWh)")
plt.title("Seasonal PV Generation Pattern")
plt.xticks(range(1, 13))
plt.tight_layout()
plt.show()
```



The data shows a strong seasonal dependence. The seasonal curve reaches roughly 0.15 kWh per half-hour in January, falls to about 0.07 in June, and rises again toward 0.15 by December.

Day-to-day variability

Example Customer

```
In [9]: # Prepare time variables
pv["day"] = pv["datetime"].dt.date
pv["hour"] = pv["datetime"].dt.hour + pv["datetime"].dt.minute / 60

# Choose example customer
example_customer = 1
customer_daily = (
    pv[pv["Customer"] == example_customer]
    .groupby(["day", "hour"])["kWh"]
    .mean()

customer_day_profile = customer_daily.groupby("hour").agg(
    mean="mean",
    std="std",
    p10=lambda x: x.quantile(0.10),
    p25=lambda x: x.quantile(0.25),
    median="median",
    p75=lambda x: x.quantile(0.75),
    p90=lambda x: x.quantile(0.90))

customer_day_profile["IQR"] = (customer_day_profile["p75"] - customer_day_profile["p25"])
customer_day_profile["CV"] = (customer_day_profile["std"] / customer_day_profile["mean"])
```

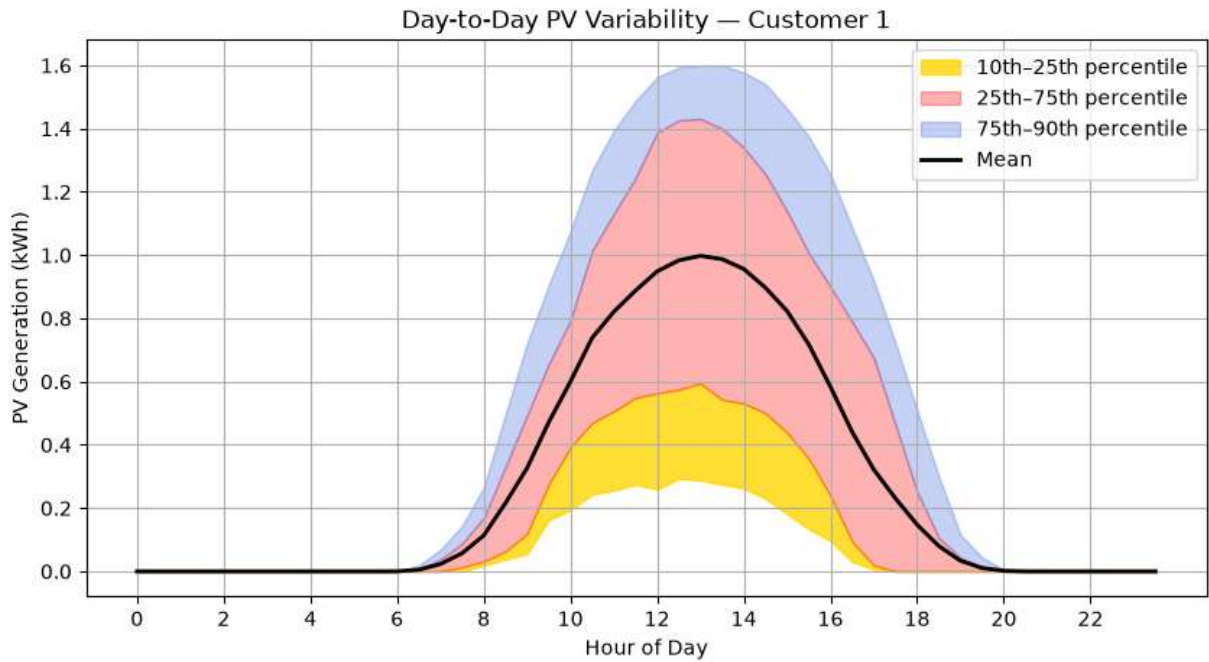
```
# Show table  
display(customer_day_profile.round(3))
```

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
0.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
1.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
1.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
2.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
2.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
3.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
3.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
4.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
4.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
5.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
5.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
6.0	0.000	0.001	0.000	0.000	0.000	0.000	0.000	0.000	6.314
6.5	0.005	0.011	0.000	0.000	0.000	0.006	0.019	0.006	2.135
7.0	0.023	0.031	0.000	0.000	0.006	0.038	0.069	0.038	1.368
7.5	0.057	0.057	0.000	0.013	0.038	0.088	0.144	0.075	1.004
8.0	0.114	0.099	0.019	0.031	0.081	0.169	0.266	0.138	0.868
8.5	0.215	0.176	0.038	0.063	0.163	0.331	0.497	0.268	0.817
9.0	0.327	0.245	0.056	0.119	0.250	0.494	0.722	0.375	0.749
9.5	0.471	0.269	0.163	0.275	0.413	0.656	0.906	0.381	0.570
10.0	0.600	0.309	0.194	0.392	0.560	0.794	1.075	0.402	0.516
10.5	0.738	0.364	0.244	0.469	0.719	1.014	1.269	0.546	0.494
11.0	0.821	0.405	0.256	0.506	0.806	1.131	1.394	0.625	0.493
11.5	0.888	0.431	0.275	0.548	0.881	1.244	1.488	0.696	0.485
12.0	0.949	0.470	0.260	0.563	0.972	1.388	1.563	0.825	0.496
12.5	0.983	0.477	0.294	0.575	1.006	1.426	1.594	0.852	0.485
13.0	0.998	0.483	0.288	0.594	1.056	1.431	1.600	0.837	0.484
13.5	0.987	0.487	0.275	0.542	1.084	1.400	1.600	0.857	0.493
14.0	0.956	0.476	0.263	0.531	1.050	1.340	1.578	0.809	0.498

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
14.5	0.897	0.463	0.231	0.500	0.975	1.256	1.541	0.756	0.516
15.0	0.823	0.450	0.181	0.438	0.878	1.138	1.463	0.700	0.547
15.5	0.718	0.429	0.134	0.356	0.700	1.006	1.375	0.650	0.598
16.0	0.584	0.419	0.097	0.238	0.469	0.900	1.256	0.662	0.718
16.5	0.441	0.400	0.031	0.094	0.294	0.790	1.088	0.696	0.907
17.0	0.322	0.364	0.006	0.019	0.119	0.675	0.919	0.656	1.132
17.5	0.231	0.290	0.000	0.000	0.044	0.463	0.722	0.463	1.255
18.0	0.148	0.204	0.000	0.000	0.006	0.252	0.510	0.252	1.384
18.5	0.080	0.125	0.000	0.000	0.000	0.106	0.300	0.106	1.557
19.0	0.033	0.056	0.000	0.000	0.000	0.044	0.116	0.044	1.664
19.5	0.011	0.021	0.000	0.000	0.000	0.013	0.044	0.013	1.989
20.0	0.002	0.005	0.000	0.000	0.000	0.000	0.006	0.000	2.846
20.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	33.106
21.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
21.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
22.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
22.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
23.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN
23.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	NaN

```
In [10]: # Plot
plt.figure(figsize=(10, 5))
plt.fill_between(customer_day_profile.index,
                 customer_day_profile["p10"],
                 customer_day_profile["p25"],
                 color="gold", alpha=0.8, label="10th-25th percentile")
plt.fill_between(customer_day_profile.index,
                 customer_day_profile["p25"],
                 customer_day_profile["p75"],
                 color="red", alpha=0.3, label="25th-75th percentile")
plt.fill_between(customer_day_profile.index,
                 customer_day_profile["p75"],
                 customer_day_profile["p90"],
                 color="royalblue", alpha=0.3, label="75th-90th percentile")
plt.plot(customer_day_profile.index,
         customer_day_profile["mean"],
         color="black", linewidth=2, label="Mean")
```

```
plt.xlabel("Hour of Day")
plt.ylabel("PV Generation (kWh)")
plt.title(f"Day-to-Day PV Variability — Customer {example_customer}")
plt.xticks(range(0, 24, 2))
plt.legend()
plt.grid()
plt.show()
```



Day-to-day variability is substantial. For the example customer, the 10th-90th percentile range becomes very wide around the middle of the day. At roughly 12:00-14:00, the mean is around 1 kWh per half-hour, while individual days range from very low generation to 1.5 kWh or more. This means that for predicting PV, time-of-day alone will not be sufficient. Weather conditions create substantial uncertainty, so variables such as temperature, irradiance, or cloud cover should improve PV estimation.

Entire System

```
In [11]: # Entire System
system_daily = (
    pv.groupby(["day", "hour"])["kWh"]
    .sum()

system_day_profile = system_daily.groupby("hour").agg(
    mean="mean",
    std="std",
    p10=lambda x: x.quantile(0.10),
    p25=lambda x: x.quantile(0.25),
    median="median",
    p75=lambda x: x.quantile(0.75),
    p90=lambda x: x.quantile(0.90))
```

```
system_day_profile["IQR"] = (system_day_profile["p75"] - system_day_profile["p25"])
system_day_profile["CV"] = (system_day_profile["std"] / system_day_profile["mean"])

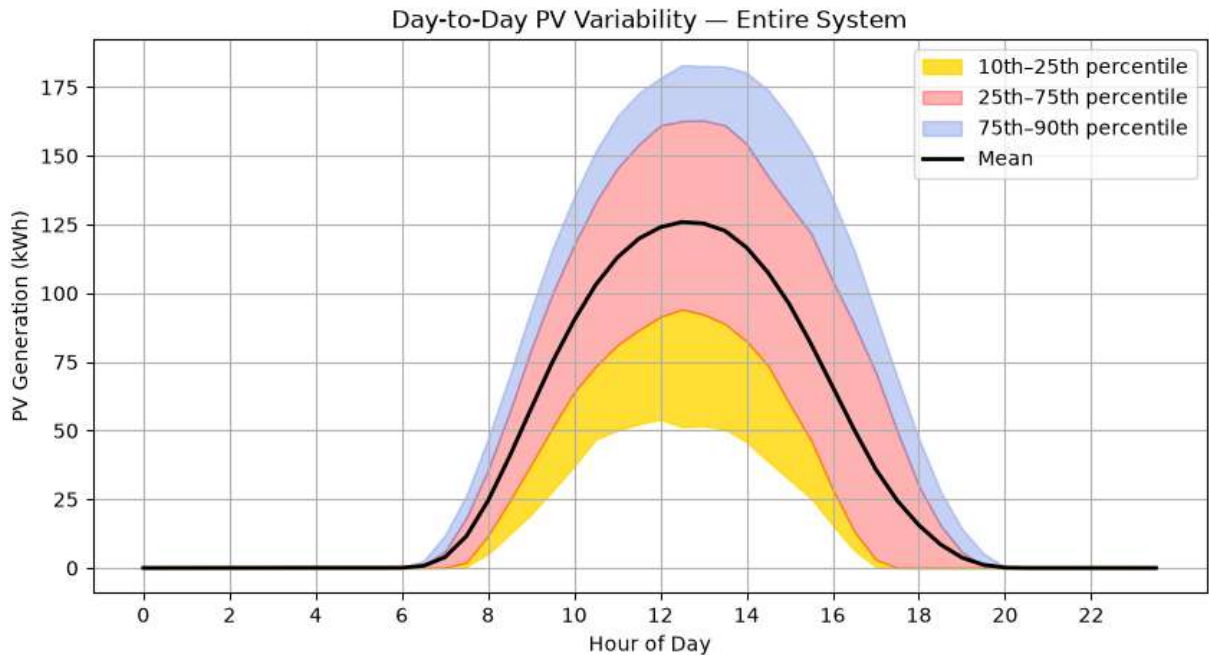
# Show table
display(system_day_profile.round(3))
```

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
0.0	0.024	0.016	0.007	0.013	0.021	0.031	0.038	0.018	0.659
0.5	0.026	0.031	0.006	0.012	0.019	0.030	0.040	0.018	1.203
1.0	0.033	0.060	0.006	0.013	0.019	0.030	0.043	0.017	1.790
1.5	0.042	0.093	0.006	0.012	0.019	0.030	0.042	0.018	2.235
2.0	0.049	0.121	0.006	0.012	0.018	0.030	0.042	0.018	2.466
2.5	0.058	0.152	0.006	0.012	0.019	0.030	0.046	0.018	2.617
3.0	0.070	0.197	0.006	0.012	0.018	0.030	0.044	0.018	2.789
3.5	0.077	0.216	0.006	0.012	0.019	0.030	0.049	0.018	2.795
4.0	0.078	0.224	0.006	0.012	0.018	0.030	0.043	0.018	2.873
4.5	0.079	0.228	0.006	0.012	0.018	0.025	0.048	0.013	2.899
5.0	0.080	0.234	0.006	0.012	0.018	0.030	0.049	0.018	2.902
5.5	0.080	0.232	0.006	0.012	0.018	0.030	0.048	0.018	2.897
6.0	0.101	0.249	0.006	0.012	0.024	0.037	0.194	0.025	2.478
6.5	0.764	1.561	0.012	0.018	0.043	0.801	2.688	0.783	2.044
7.0	3.907	5.360	0.020	0.059	1.244	6.133	11.984	6.074	1.372
7.5	11.619	10.856	0.602	2.120	8.814	18.556	26.810	16.436	0.934
8.0	24.461	16.432	5.229	11.826	21.053	35.815	47.252	23.990	0.672
8.5	40.612	21.971	12.423	24.413	37.446	56.947	70.141	32.534	0.541
9.0	58.051	27.610	19.610	37.710	56.802	79.732	94.170	42.023	0.476
9.5	75.176	32.421	28.258	51.222	76.645	100.299	116.472	49.077	0.431
10.0	90.195	36.468	37.204	64.011	95.580	117.816	135.411	53.805	0.404
10.5	103.107	39.848	46.995	73.443	111.251	133.349	151.846	59.906	0.386
11.0	112.983	42.719	50.127	81.121	122.669	145.641	164.538	64.520	0.378
11.5	119.884	44.828	52.453	86.692	130.281	154.224	172.974	67.532	0.374
12.0	124.037	46.225	54.156	91.483	134.454	161.190	178.471	69.707	0.373
12.5	125.928	47.248	51.408	94.136	135.610	162.721	183.090	68.585	0.375
13.0	125.422	48.012	51.732	92.305	134.464	162.903	182.772	70.598	0.383
13.5	122.745	48.010	50.512	88.821	130.235	161.134	182.606	72.312	0.391
14.0	116.644	48.114	45.922	82.487	120.974	154.358	180.291	71.871	0.412

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
14.5	107.531	47.519	38.894	73.504	108.754	142.523	173.974	69.019	0.442
15.0	95.848	46.931	32.109	59.430	93.462	131.916	163.932	72.486	0.490
15.5	81.448	46.021	25.380	46.240	72.783	121.713	151.595	75.473	0.565
16.0	65.909	44.463	15.631	28.483	54.471	104.364	134.358	75.881	0.675
16.5	50.281	42.015	6.495	13.229	35.144	88.594	115.520	75.366	0.836
17.0	35.906	36.622	0.526	2.996	18.360	71.061	92.298	68.065	1.020
17.5	24.470	28.442	0.019	0.060	6.502	49.596	69.284	49.536	1.162
18.0	15.587	19.457	0.013	0.024	0.764	29.761	46.992	29.737	1.248
18.5	8.589	11.468	0.012	0.024	0.064	15.558	27.987	15.534	1.335
19.0	3.761	5.877	0.012	0.020	0.049	5.751	14.262	5.731	1.562
19.5	1.143	2.250	0.012	0.019	0.032	0.764	5.266	0.745	1.969
20.0	0.138	0.327	0.012	0.018	0.025	0.043	0.405	0.025	2.378
20.5	0.023	0.012	0.008	0.013	0.024	0.030	0.038	0.017	0.506
21.0	0.024	0.012	0.010	0.015	0.024	0.031	0.042	0.016	0.505
21.5	0.023	0.012	0.008	0.014	0.024	0.031	0.038	0.017	0.504
22.0	0.023	0.012	0.007	0.013	0.022	0.031	0.038	0.018	0.509
22.5	0.023	0.011	0.008	0.014	0.024	0.030	0.038	0.016	0.499
23.0	0.022	0.012	0.007	0.013	0.020	0.030	0.038	0.017	0.539
23.5	0.021	0.011	0.007	0.013	0.019	0.028	0.037	0.015	0.534

```
In [12]: # Plot
plt.figure(figsize=(10, 5))
plt.fill_between(system_day_profile.index,
                 system_day_profile["p10"],
                 system_day_profile["p25"],
                 color="gold", alpha=0.8, label="10th-25th percentile")
plt.fill_between(system_day_profile.index,
                 system_day_profile["p25"],
                 system_day_profile["p75"],
                 color="red", alpha=0.3, label="25th-75th percentile")
plt.fill_between(system_day_profile.index,
                 system_day_profile["p75"],
                 system_day_profile["p90"],
                 color="royalblue", alpha=0.3, label="75th-90th percentile")
plt.plot(system_day_profile.index,
         system_day_profile["mean"],
         color="black", linewidth=2, label="Mean")
```

```
plt.xlabel("Hour of Day")
plt.ylabel("PV Generation (kWh)")
plt.title("Day-to-Day PV Variability — Entire System")
plt.xticks(range(0, 24, 2))
plt.legend()
plt.grid()
plt.show()
```



For the entire system, the mean midday generation is roughly 120-130 kWh per half-hour, but the percentile range is still broad. The curve is smoother, as aggregation decreases the relative variation. This suggests that aggregated BTM PV should be easier to estimate than individual-house PV.

Customer-to-Customer Variability

```
In [13]: # Average profile for each customer
customer_profiles = (
    pv.groupby(["Customer", "hour"])["kWh"]
    .mean()
    .reset_index()

customer_variability = customer_profiles.groupby("hour")["kWh"].agg(
    mean="mean",
    std="std",
    p10=lambda x: x.quantile(0.10),
    p25=lambda x: x.quantile(0.25),
    median="median",
    p75=lambda x: x.quantile(0.75),
    p90=lambda x: x.quantile(0.90))

customer_variability["IQR"] = (customer_variability["p75"] - customer_variability["p25"])
customer_variability["CV"] = (customer_variability["std"] / customer_variability["mean"])
```

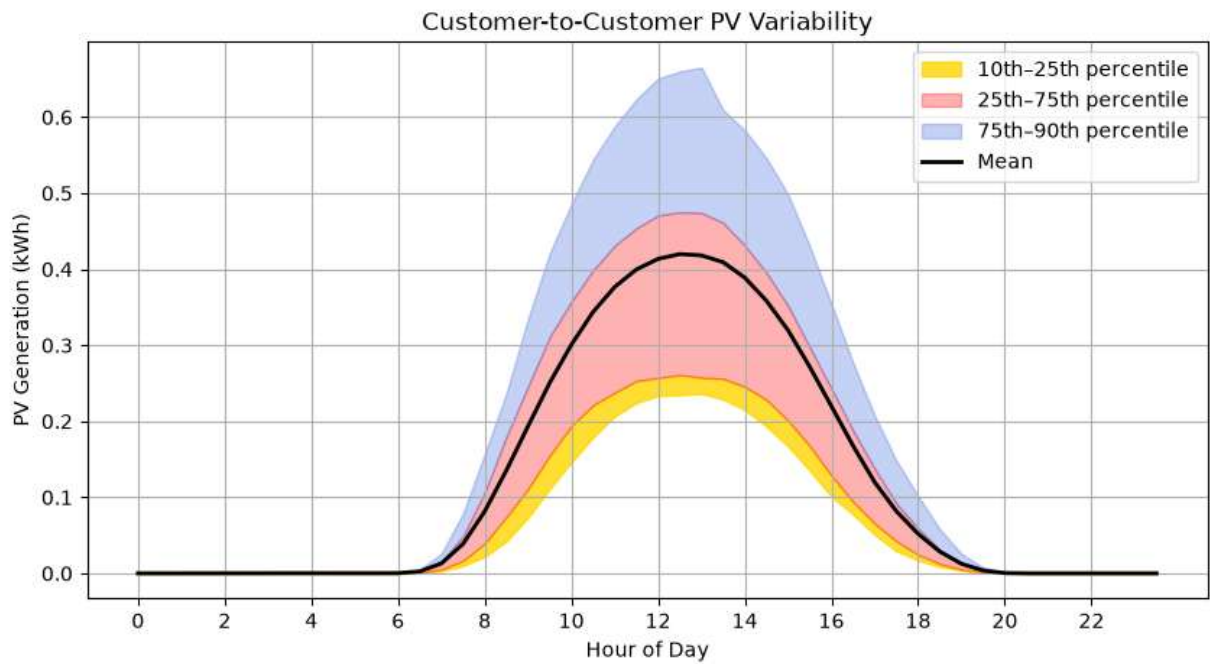
```
display(customer_variability.round(3))
```

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.526
0.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	3.757
1.0	0.000	0.001	0.000	0.000	0.000	0.000	0.000	0.000	6.405
1.5	0.000	0.001	0.000	0.000	0.000	0.000	0.000	0.000	8.797
2.0	0.000	0.002	0.000	0.000	0.000	0.000	0.000	0.000	9.879
2.5	0.000	0.002	0.000	0.000	0.000	0.000	0.000	0.000	10.343
3.0	0.000	0.002	0.000	0.000	0.000	0.000	0.000	0.000	10.157
3.5	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	10.231
4.0	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	10.590
4.5	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	10.769
5.0	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	11.019
5.5	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	11.050
6.0	0.000	0.003	0.000	0.000	0.000	0.000	0.000	0.000	8.673
6.5	0.003	0.005	0.000	0.001	0.002	0.003	0.005	0.002	1.790
7.0	0.013	0.016	0.003	0.005	0.009	0.015	0.026	0.010	1.206
7.5	0.039	0.039	0.010	0.017	0.029	0.049	0.079	0.032	1.008
8.0	0.082	0.071	0.022	0.039	0.063	0.106	0.157	0.067	0.870
8.5	0.135	0.105	0.042	0.074	0.114	0.180	0.237	0.106	0.775
9.0	0.194	0.140	0.074	0.111	0.164	0.246	0.335	0.135	0.724
9.5	0.251	0.173	0.111	0.155	0.207	0.311	0.421	0.156	0.691
10.0	0.301	0.198	0.147	0.194	0.243	0.357	0.486	0.164	0.659
10.5	0.344	0.221	0.179	0.221	0.280	0.399	0.544	0.178	0.644
11.0	0.377	0.239	0.207	0.238	0.311	0.431	0.588	0.193	0.635
11.5	0.400	0.256	0.224	0.253	0.329	0.454	0.624	0.201	0.640
12.0	0.414	0.269	0.234	0.257	0.342	0.471	0.651	0.214	0.651
12.5	0.420	0.276	0.235	0.261	0.359	0.475	0.660	0.214	0.657
13.0	0.418	0.276	0.236	0.257	0.359	0.474	0.665	0.217	0.660
13.5	0.409	0.272	0.229	0.256	0.350	0.461	0.609	0.205	0.665
14.0	0.389	0.261	0.215	0.245	0.328	0.431	0.583	0.186	0.672

	mean	std	p10	p25	median	p75	p90	IQR	CV
hour									
14.5	0.359	0.244	0.193	0.228	0.304	0.395	0.546	0.167	0.681
15.0	0.320	0.222	0.167	0.201	0.266	0.352	0.498	0.151	0.696
15.5	0.272	0.195	0.135	0.167	0.227	0.297	0.430	0.130	0.720
16.0	0.220	0.163	0.101	0.128	0.185	0.243	0.354	0.114	0.742
16.5	0.168	0.127	0.078	0.094	0.138	0.188	0.277	0.094	0.759
17.0	0.120	0.096	0.051	0.065	0.096	0.137	0.207	0.072	0.799
17.5	0.082	0.071	0.029	0.042	0.066	0.092	0.148	0.050	0.866
18.0	0.052	0.048	0.017	0.025	0.038	0.059	0.102	0.035	0.925
18.5	0.029	0.029	0.009	0.012	0.020	0.031	0.058	0.018	1.009
19.0	0.013	0.014	0.004	0.005	0.008	0.013	0.026	0.008	1.102
19.5	0.004	0.005	0.001	0.001	0.002	0.004	0.008	0.003	1.202
20.0	0.000	0.001	0.000	0.000	0.000	0.001	0.001	0.000	1.299
20.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.382
21.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.336
21.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.365
22.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.395
22.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.318
23.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.401
23.5	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.310

```
In [14]: # Plot
plt.figure(figsize=(10, 5))
plt.fill_between(customer_variability.index,
                 customer_variability["p10"],
                 customer_variability["p25"],
                 color="gold", alpha=0.8, label="10th-25th percentile")
plt.fill_between(customer_variability.index,
                 customer_variability["p25"],
                 customer_variability["p75"],
                 color="red", alpha=0.3, label="25th-75th percentile")
plt.fill_between(customer_variability.index,
                 customer_variability["p75"],
                 customer_variability["p90"],
                 color="royalblue", alpha=0.3, label="75th-90th percentile")
plt.plot(customer_variability.index,
         customer_variability["mean"],
         color="black", linewidth=2, label="Mean")
```

```
plt.xlabel("Hour of Day")
plt.ylabel("PV Generation (kWh)")
plt.title("Customer-to-Customer PV Variability")
plt.xticks(range(0, 24, 2))
plt.legend()
plt.grid()
plt.show()
```



The customer-to-customer variability plot has a much narrower band than the day-to-day system variation plot. Around midday, the mean is about 0.4 kWh per half-hour, with the central 50% and 10-90% ranges comparatively compact. This suggests that weather-driven variation is a larger source of uncertainty than customer-to-customer differences.

Load Data

```
In [1]: import requests
import pandas as pd

# Approximate location for postcode 2076
lat = -33.72
lon = 151.12

url = "https://archive-api.open-meteo.com/v1/archive"

params = {
    "latitude": lat,
    "longitude": lon,
    "start_date": "2010-07-01",
    "end_date": "2013-06-30",
    "hourly": "temperature_2m,shortwave_radiation,precipitation",
    "models": "era5",
    "timezone": "Australia/Sydney",
    "temperature_unit": "celsius",
    "precipitation_unit": "mm",
}

response = requests.get(url, params=params)
response.raise_for_status()

weather = pd.DataFrame(response.json()["hourly"])
weather["time"] = pd.to_datetime(weather["time"])

weather.head()
```

```
Out[1]:
```

	time	temperature_2m	shortwave_radiation	precipitation
0	2010-07-01 00:00:00	3.6	0.0	0.0
1	2010-07-01 01:00:00	3.3	0.0	0.0
2	2010-07-01 02:00:00	2.2	0.0	0.0
3	2010-07-01 03:00:00	2.0	0.0	0.0
4	2010-07-01 04:00:00	1.8	0.0	0.0

Check data quality

```
In [2]: import matplotlib.pyplot as plt

# Coverage and time resolution
weather['time'] = pd.to_datetime(weather['time'])
weather = weather.sort_values('time')
```

```
print("Coverage:", weather["time"].min(), "to", weather["time"].max())
print("Rows:", len(weather))
print("Duplicate timestamps:", weather["time"].duplicated().sum())

# Time gaps
gaps = weather["time"].diff().dropna()
print("\nTime intervals:")
print(gaps.value_counts().head())

print("\nUnexpected intervals:")
print(gaps[gaps != pd.Timedelta(hours=1)].value_counts())

# Missingness
print("\nMissing values:")
print(weather.isna().sum())
print("\nMissing %:")
print((weather.isna().mean() * 100).round(2))

# Distributions and summary statistics
variables = ["temperature_2m", "shortwave_radiation", "precipitation"]

print("\nSummary statistics:")
print(weather[variables].describe().round(2))
```

Coverage: 2010-07-01 00:00:00 to 2013-06-30 23:00:00

Rows: 26304

Duplicate timestamps: 0

Time intervals:

time

0 days 01:00:00 26303

Name: count, dtype: int64

Unexpected intervals:

Series([], Name: count, dtype: int64)

Missing values:

time 0

temperature_2m 0

shortwave_radiation 0

precipitation 0

dtype: int64

Missing %:

time 0.0

temperature_2m 0.0

shortwave_radiation 0.0

precipitation 0.0

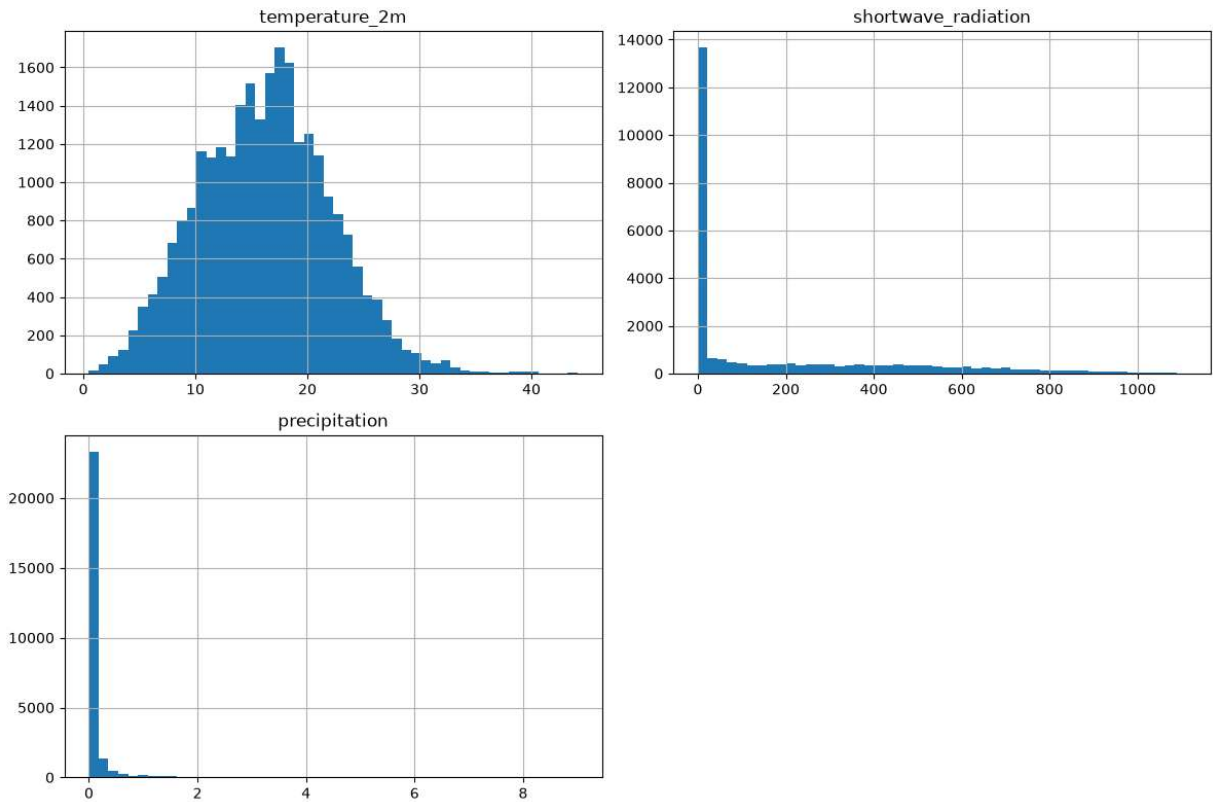
dtype: float64

Summary statistics:

	temperature_2m	shortwave_radiation	precipitation
count	26304.00	26304.00	26304.00
mean	16.20	187.30	0.09
std	5.87	262.58	0.38
min	0.50	0.00	0.00
25%	12.00	0.00	0.00
50%	16.30	7.00	0.00
75%	20.10	347.00	0.00
max	44.10	1111.00	9.00

- **coverage:** Exactly 3 years, from July 1, 2010 through June 30, 2013
- **Completeness:** 26,304 hourly observations, with no missing values in any of the three variables.
- **Time Consistency:** No gaps or duplicate timestamps, every interval is exactly 1 hour.
- **Temperature:** Mean 16.2°C, range 0.5-44.1°C. The 44.1°C maximum is high but plausible for Sydney.
- **Solar Radiation:** Mean 187 W/m^2 , with strong day/night signal indicated by the median of only 7 W/m^2 and maximum of 1,111 W/m^2 . Broadly plausible.
- **Precipitation:** Mean 0.09 mm/hour, median 0, maximum 9 mm/hour. The many zeros are expected because rainfall is intermittent.

```
In [3]: # Histograms
weather[variables].hist(bins=50, figsize=(12, 8))
plt.tight_layout()
plt.show()
```



- **X-axis:** The measured weather value.
- **Y-axis:** Number of hourly observations falling within each value range.
- **Temperature:** Bell-shaped, centered around 15-20C. The values range from 0-44C, consistent with seasonal temperature variation.
- **Shortwave Radiation:** The large spike near 0 W/m² represents nighttime and low-light hours. Values extend to 1,100 W/m², with fewer observations at high radiation levels. This strongly right-skewed shape is expected.
- **Precipitation:** The overwhelming majority of hours have no precipitation. This distribution is expected.

Format datetimes to match Ausgrid Data

```
In [4]: # Format datetime to match Ausgrid data
weather["datetime"] = pd.to_datetime(weather["time"]).dt.tz_localize(None)
weather = weather.drop(columns="time")

# Interpolate hourly data to 30-minute resolution
weather = (
    weather.set_index("datetime")
    .resample("30min")
    .interpolate()
    .reset_index()
```

```
# Show new table  
weather.head()
```

Out[4]:

	datetime	temperature_2m	shortwave_radiation	precipitation
0	2010-07-01 00:00:00	3.60	0.0	0.0
1	2010-07-01 00:30:00	3.45	0.0	0.0
2	2010-07-01 01:00:00	3.30	0.0	0.0
3	2010-07-01 01:30:00	2.75	0.0	0.0
4	2010-07-01 02:00:00	2.20	0.0	0.0

Save Data

```
In [6]: import os  
  
base_dir = os.getcwd()  
data_dir = os.path.join(base_dir, 'Ausgrid_solar_home_data')  
csv_save_filepath = os.path.join(data_dir, 'weather_data_processed.csv')  
  
weather.to_csv(csv_save_filepath, index=False)
```

In []:

Combine Datasets

```
In [1]: import os
import pandas as pd

base_dir = os.getcwd()
data_dir = os.path.join(base_dir, 'Ausgrid_solar_home_data')

# Load processed datasets
ausgrid = pd.read_csv(os.path.join(data_dir, "ausgrid_processed.csv"))
weather = pd.read_csv(os.path.join(data_dir, "weather_data_processed.csv"))

# Combine datasets
df = ausgrid.merge(weather, on="datetime", how="left")
df.head()
```

```
Out[1]:
```

	Customer	Generator Capacity	Postcode	Consumption Category	datetime	kWh	Unnamed: 0	temperature
0	1	3.78	2076	CL	2010-07-01 00:00:00	1.075	0.0	
1	1	3.78	2076	CL	2010-07-01 00:30:00	1.250	1.0	
2	1	3.78	2076	CL	2010-07-01 01:00:00	1.244	2.0	
3	1	3.78	2076	CL	2010-07-01 01:30:00	1.256	3.0	
4	1	3.78	2076	CL	2010-07-01 02:00:00	0.744	4.0	

The Ausgrid dataset uses three consumption categories:

- GC: General Consumption
- CL: Controlled Load
- GG: Gross Generation (solar PV generation)

Consider a real distribution system. Net load is monitored for each customer, but residential solar generation may not be directly measured. The size, orientation, and other characteristics of the solar installations may also be unknown. Our goal is to disaggregate

net load into base load and PV generation. Net load is given by $P_{\text{net}} = P_{\text{load}} - P_{\text{PV}}$ where $P_{\text{load}} = GC + CL$. We can do this by predicting P_{PV} .

A common simplified equation for PV power output is:

$$P_{\text{PV}} = P_{\text{rated}} \frac{G}{G_{\text{STC}}} [1 + \gamma(T_{\text{cell}} - T_{\text{STC}})]$$

- P_{PV} = PV electrical power output (kW).
- P_{rated} = rated/nominal PV system capacity (kW).
- G = incident solar irradiance on the panels (W/m^2).
- G_{STC} = reference irradiance at standard test conditions, typically 1000 W/m^2 .
- T_{cell} = PV cell temperature (C)
- T_{STC} = reference cell temperature, typically 25 C.
- γ = temperature coefficient of power (per C), normally negative.

Irradiance primarily determines how much power is available, where cell temperature modifies the output. Solar irradiance is the solar power received per unit area, typically measured in W/m^2 . For PV modeling, it ideally refers to the radiation incident on the panel surface. Shortwave radiation is a broader atmospheric measurement of incoming radiation in the shortwave portion of the spectrum, also measured in W/m^2 . In our dataset, `shortwave_radiation` can serve as a proxy for the solar irradiance available to the PV system.

The rated PV system capacity P_{rated} is unknown. The incident solar radiation G can be estimated from `shortwave_radiation` and the position of the sun, which can in turn be estimated from the `hour of day` and the `geographical location` of the system. A useful starting point to model the irradiance incident on the PV panels as:

$$G(t) = S(t)f(\theta_{\text{sun}}(t, \phi, \lambda), \beta)$$

- $G(t)$ = irradiance incident on the PV panels at time t (W/m^2)
- $S(t)$ = measured shortwave radiation (W/m^2)
- t = time of day/date.
- ϕ, λ = latitude and longitude of the PV system.
- θ_{sun} = solar position.
- β = panel tilt, which is unknown and must be estimated.
- $f(\cdot)$ = function describing how solar position and panel orientation determine the radiation incident on the panel.

$$\theta_{\text{sun}} = \sin \phi \sin \delta + \cos \phi \cos \delta \cos H$$

- δ = solar declination, determined by the date.
- H = solar hour angle, determined by time of day.

$$\delta = 23.45 \text{ degree} \sin\left(\frac{360 \text{ degree}}{365}(284 + n)\right)$$

- n = day of year (1-365)

$$f(\theta_{\text{sun}}, \beta) = \max(0, \cos(\theta_{\text{sun}} - \beta))$$

The expression for P_{PV} becomes:

$$P_{\text{PV}} = P_{\text{rated}} \times \frac{S(t) \max(0, \cos(\theta_{\text{sun}} - \beta))}{G_{\text{STC}}} [1 + \gamma(T_{\text{cell}} - T_{\text{STC}})]$$

Replace P_{rated} with parameter α which represents unknown characteristics of the PV system affecting maximum output, such as area and number of panels. The parameter β represents the unknown angle of the panels. The parameter γ represents the unknown temperature coefficient of power.

Calculate Sun Position

```
In [2]: import numpy as np
import pandas as pd

lat = 33.9 # system latitude (degrees)
lon = 151.2 # system longitude (degrees)

# Convert string to datetime type
df["datetime"] = pd.to_datetime(df["datetime"])

sun = pd.DataFrame({"datetime": df["datetime"]})
sun["day_of_year"] = sun["datetime"].dt.dayofyear
sun["hour"] = sun["datetime"].dt.hour + sun["datetime"].dt.minute / 60

# Solar declination
sun["declination_angle"] = np.radians(23.45 * np.sin(np.radians((360 / 365) * (284
```

```
# Solar hour angle (simplified: 15° per hour from solar noon)
hour_angle = np.radians(15 * (sun['hour'] - 12))

# Solar zenith angle
lat_rad = np.radians(lat)
sun["sun_position"] = np.degrees(np.arccos(
    np.sin(lat_rad) * np.sin(sun["declination_angle"]) +
    np.cos(lat_rad) * np.cos(sun["declination_angle"]) * np.cos(hour_angle)))

# Make sure sun does not have duplicate datetime values
sun = sun[["datetime", "sun_position"]].drop_duplicates("datetime")

sun.head()
```

```
Out[2]:
```

	datetime	sun_position
0	2010-07-01 00:00:00	122.979516
1	2010-07-01 00:30:00	122.534587
2	2010-07-01 01:00:00	121.220214
3	2010-07-01 01:30:00	119.093277
4	2010-07-01 02:00:00	116.236036

Append sun position to original dataset

```
In [3]: df = df.merge(
    sun[["datetime", "sun_position"]],
    on="datetime",
    how="left")

df.head()
```

```
Out[3]:
```

	Customer	Generator Capacity	Postcode	Consumption Category	datetime	kWh	Unnamed: 0	temperature
0	1	3.78	2076	CL	2010-07-01 00:00:00	1.075	0.0	
1	1	3.78	2076	CL	2010-07-01 00:30:00	1.250	1.0	
2	1	3.78	2076	CL	2010-07-01 01:00:00	1.244	2.0	
3	1	3.78	2076	CL	2010-07-01 01:30:00	1.256	3.0	
4	1	3.78	2076	CL	2010-07-01 02:00:00	0.744	4.0	

Our model is simplified to: $P_{PV}(t) = C \times S(t) \times \theta_{\text{sun}}(t) \frac{1 + \gamma(T_{\text{Cell}}(t) - T_{\text{STC}})}{G_{\text{STC}}}$

- $S(t)$ is `shortwave_radiation`.
- $\theta_{\text{sun}}(t)$ is the position of the sun
- $T_{\text{cell}}(t)$ is estimated as `temperature_2m`.
- T_{STC} and G_{STC} are known constants.
- C is a per-customer unknown parameter accounting for PV system characteristics such as total generating capacity and panel tilt angle.

- γ is a per-customer unknown parameter representing the PV system's temperature coefficient of power (per C).

Parameter Estimation

Modify the dataset

```
In [4]: df["Pload"] = df["kWh"].where(df["Consumption Category"].isin(["GC", "CL"]), 0)
df["PPV"] = df["kWh"].where(df["Consumption Category"] == "GG", 0)
df["Pnet"] = df["Pload"] - df["PPV"]
df.drop(columns=["Consumption Category", "kWh"], inplace=True)
df.head()
```

```
Out[4]:
```

	Customer	Generator Capacity	Postcode	datetime	Unnamed: 0	temperature_2m	shortwave_radia
0	1	3.78	2076	2010-07-01 00:00:00	0.0	3.60	
1	1	3.78	2076	2010-07-01 00:30:00	1.0	3.45	
2	1	3.78	2076	2010-07-01 01:00:00	2.0	3.30	
3	1	3.78	2076	2010-07-01 01:30:00	3.0	2.75	
4	1	3.78	2076	2010-07-01 02:00:00	4.0	2.20	

Remove Missing Data

```
In [5]: # Remove the unnamed column
df = df.drop(columns=["Unnamed: 0"])

# Remove rows containing any missing values
df = df.dropna()

# Count NaNs in each column
display(df.isna().sum())
```

```

Customer          0
Generator Capacity 0
Postcode          0
datetime          0
temperature_2m     0
shortwave_radiation 0
precipitation      0
sun_position       0
Pload             0
PPV               0
Pnet              0
dtype: int64

```

Split Training and Testing Data

```

In [6]: # Randomly split each customer's data 80/20
train_df = (
    df.groupby("Customer", group_keys=False)
      .sample(frac=0.8, random_state=42))

# Everything not selected for training goes into testing
test_df = df.drop(train_df.index)

# Optional: reset indices
train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

```

Perform Parameter Estimation

```

In [7]: import numpy as np
from scipy.optimize import least_squares

G_STC = 1000
T_STC = 25

def residuals(params, S, theta_sun, T, P):
    alpha, beta, gamma = params

    # Irradiance adjustment for panel tilt
    incidence_factor = np.maximum(0, np.cos(np.radians(theta_sun - beta)))

    # Predicted PV energy (kWh per 30-minute interval)
    P_hat = (
        alpha
        * S / G_STC
        * incidence_factor
        * (1 + gamma * (T - T_STC))
        * 0.5)

    return P_hat - P

results = []

```

```

for customer, group in train_df.groupby("Customer"):
    S = group["shortwave_radiation"].values
    theta_sun = group["sun_position"].values
    T = group["temperature_2m"].values
    P = group["PPV"].values

    # Initial guesses: capacity, panel tilt, temperature coefficient
    initial_guess = [
        group["Generator Capacity"].iloc[0],
        30,
        -0.004]

    # Estimate alpha, beta, and gamma
    fit = least_squares(
        residuals,
        x0=initial_guess,
        bounds=(
            [0, 0, -0.02],
            [np.inf, 90, 0.02]),
        args=(S, theta_sun, T, P))

    results.append([
        customer,
        fit.x[0],
        fit.x[1],
        fit.x[2]])

parameters = pd.DataFrame(
    results,
    columns=["Customer", "alpha", "beta", "gamma"])

parameters.head()

```

Out[7]:

	Customer	alpha	beta	gamma
0	1	1.119017	35.992530	0.006523
1	2	0.481045	25.803825	-0.017665
2	3	0.338140	31.152128	-0.001233
3	4	0.270474	35.361862	-0.007458
4	5	0.265757	32.845339	-0.007325

Train Results

In [8]: `display(parameters.columns)`

Index(['Customer', 'alpha', 'beta', 'gamma'], dtype='str')

In [9]: `# Remove existing parameter columns`
`train_df = train_df.drop(`
 `columns=["alpha", "beta", "gamma"],`
 `errors="ignore")`

```

# Add updated parameters
train_df = train_df.merge(
    parameters[["Customer", "alpha", "beta", "gamma"]],
    on="Customer",
    how="left")

# Remove duplicate customer-datetime observations
train_df = train_df.drop_duplicates(subset=["Customer", "datetime"])

# Sort chronologically within each customer
train_df = train_df.sort_values(["Customer", "datetime"]).reset_index(drop=True)

train_df.head()

```

Out[9]:

	Customer	Generator Capacity	Postcode	datetime	temperature_2m	shortwave_radiation	precip
0	1	3.78	2076	2010-07-01 00:00:00	3.60	0.0	
1	1	3.78	2076	2010-07-01 00:30:00	3.45	0.0	
2	1	3.78	2076	2010-07-01 01:00:00	3.30	0.0	
3	1	3.78	2076	2010-07-01 01:30:00	2.75	0.0	
4	1	3.78	2076	2010-07-01 02:00:00	2.20	0.0	

```

In [10]: # Panel incidence factor
incidence_factor = np.maximum(0, np.cos(np.radians(train_df["sun_position"] - train_

# Predict PV generation
train_df["PPV_pred"] = (
    train_df["alpha"]
    * train_df["shortwave_radiation"] / 1000
    * incidence_factor
    * (1 + train_df["gamma"] * (train_df["temperature_2m"] - 25))
    * 0.5)

# Compare predicted vs. true PV generation
comparison = train_df[["Customer", "PPV", "PPV_pred"]]
comparison.head()

```

Out[10]:

	Customer	PPV	PPV_pred
0	1	0.0	0.0
1	1	0.0	0.0
2	1	0.0	0.0
3	1	0.0	0.0
4	1	0.0	0.0

```
In [11]: # Plot
import matplotlib.pyplot as plt
plt.rcParams["agg.path.chunksize"] = 10000

customer = train_df["Customer"].iloc[0]
plot_df = (
    train_df[train_df["Customer"] == customer]
    .sort_values("datetime")

plot_df.head()
```

Out[11]:

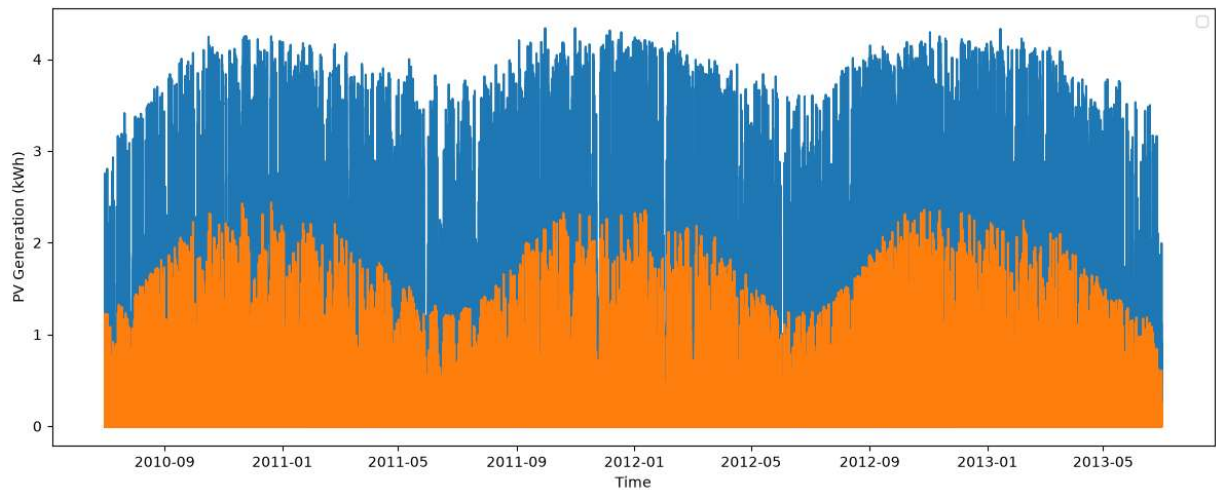
	Customer	Generator Capacity	Postcode	datetime	temperature_2m	shortwave_radiation	precip
0	1	3.78	2076	2010-07-01 00:00:00	3.60	0.0	
1	1	3.78	2076	2010-07-01 00:30:00	3.45	0.0	
2	1	3.78	2076	2010-07-01 01:00:00	3.30	0.0	
3	1	3.78	2076	2010-07-01 01:30:00	2.75	0.0	
4	1	3.78	2076	2010-07-01 02:00:00	2.20	0.0	

```
In [12]: plt.figure(figsize=(12, 5))
plt.plot(train_df["datetime"], train_df["PPV"])
plt.plot(train_df["datetime"], train_df["PPV_pred"])
# plt.plot(train_df["datetime"], train_df["PPV"], Label="True PPV")
# plt.plot(train_df["datetime"], train_df["PPV_pred"], Label="Predicted PPV")
plt.xlabel("Time")
plt.ylabel("PV Generation (kWh)")
plt.legend(loc="upper right")
```

```
plt.tight_layout()
plt.show()
```

C:\Users\codett\AppData\Local\Temp\ipykernel_12420\777822410.py:8: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
plt.legend(loc="upper right")
```



The results on the training data are showing systematic offset. Our first tpe is to check the scale of `alpha` versus `Generator Capacity`. If `alpha` is consistently smaller than actual generating capacity, that would explain the offset.

```
In [13]: check = train_df[["Customer", "Generator Capacity"]].drop_duplicates(
        "Customer").merge(parameters, on="Customer")

check["alpha_ratio"] = check["alpha"] / check["Generator Capacity"]
check[["Generator Capacity", "alpha", "alpha_ratio"]].describe()
```

```
Out[13]:
```

	Generator Capacity	alpha	alpha_ratio
count	300.000000	300.000000	300.000000
mean	1.683300	0.602797	0.356433
std	1.036846	0.412600	0.080065
min	1.000000	0.202756	0.174844
25%	1.020000	0.376698	0.292587
50%	1.500000	0.488990	0.339769
75%	2.000000	0.686975	0.422857
max	9.990000	4.509278	0.624316

The next useful diagnostic is to compare the estimated β across customers.

```
In [14]: display(parameters["beta"].describe())
parameters[["Customer", "beta"]].sort_values("beta")
```

```
print("At 0°:", (parameters["beta"] == 0).sum())
print("At 90°:", (parameters["beta"] == 90).sum())
```

```
count    300.000000
mean      35.282534
std       12.782116
min        7.400015
25%       28.203961
50%       33.278948
75%       39.947769
max       90.000000
Name: beta, dtype: float64
At 0°: 0
At 90°: 0
```

The estimated β values look reasonable. No estimates hit either bound (0\degree or 90\degree). The mean angle is 35.3\degree and most panels fall within 28.2\degree – 39.9\degree. This is consistent with the model finding a meaningful tilt parameter.

Finally, we check γ .

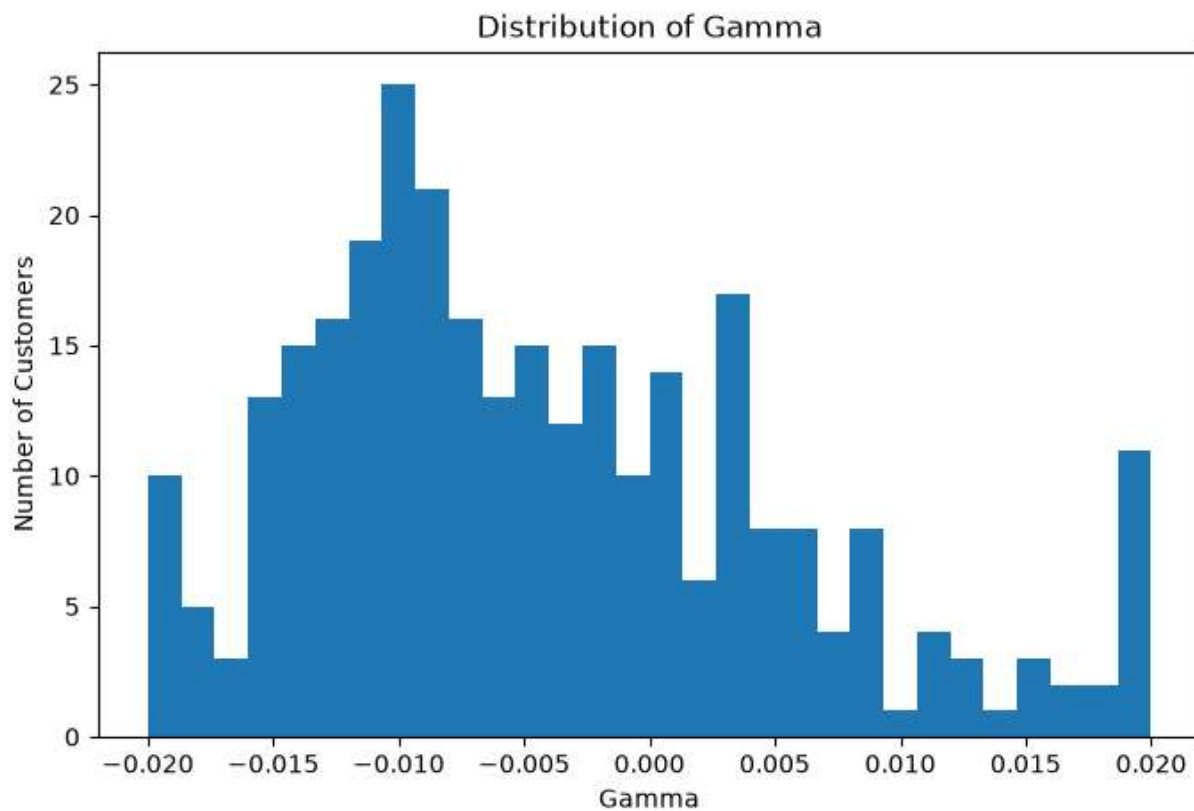
```
In [17]: display(parameters["gamma"].describe())
          parameters[["Customer", "gamma"]].sort_values("gamma")
          print("Near -0.02:", (parameters["gamma"] <= -0.01999).sum())
          print("Near 0.02:", (parameters["gamma"] >= 0.01999).sum())
```

```
count    300.000000
mean     -0.004127
std       0.009537
min      -0.020000
25%      -0.011088
50%      -0.006121
75%       0.002002
max       0.020000
Name: gamma, dtype: float64
Near -0.02: 8
Near 0.02: 11
```

19 out of 300 customers (~6.3%) have γ essentially at an imposed bound. For these customers, the optimizer may be using the temperature coefficient to compensate for limitations elsewhere in the model.

```
In [19]: import matplotlib.pyplot as plt

          plt.figure(figsize=(8, 5))
          plt.hist(parameters["gamma"], bins=30)
          plt.xlabel("Gamma")
          plt.ylabel("Number of Customers")
          plt.title("Distribution of Gamma")
          plt.show()
```



```
In [20]: # Prediction error for each observation
train_df["error"] = train_df["PPV_pred"] - train_df["PPV"]
train_df["abs_error"] = train_df["error"].abs()

# Customer-level table
error_by_customer = (
    train_df.groupby("Customer")
    .agg(
        gamma=("gamma", "first"),
        mean_error=("error", "mean"),
        MAE=("abs_error", "mean")
    ).reset_index()

# View table sorted by gamma
error_by_customer.sort_values("gamma")
```

Out[20]:

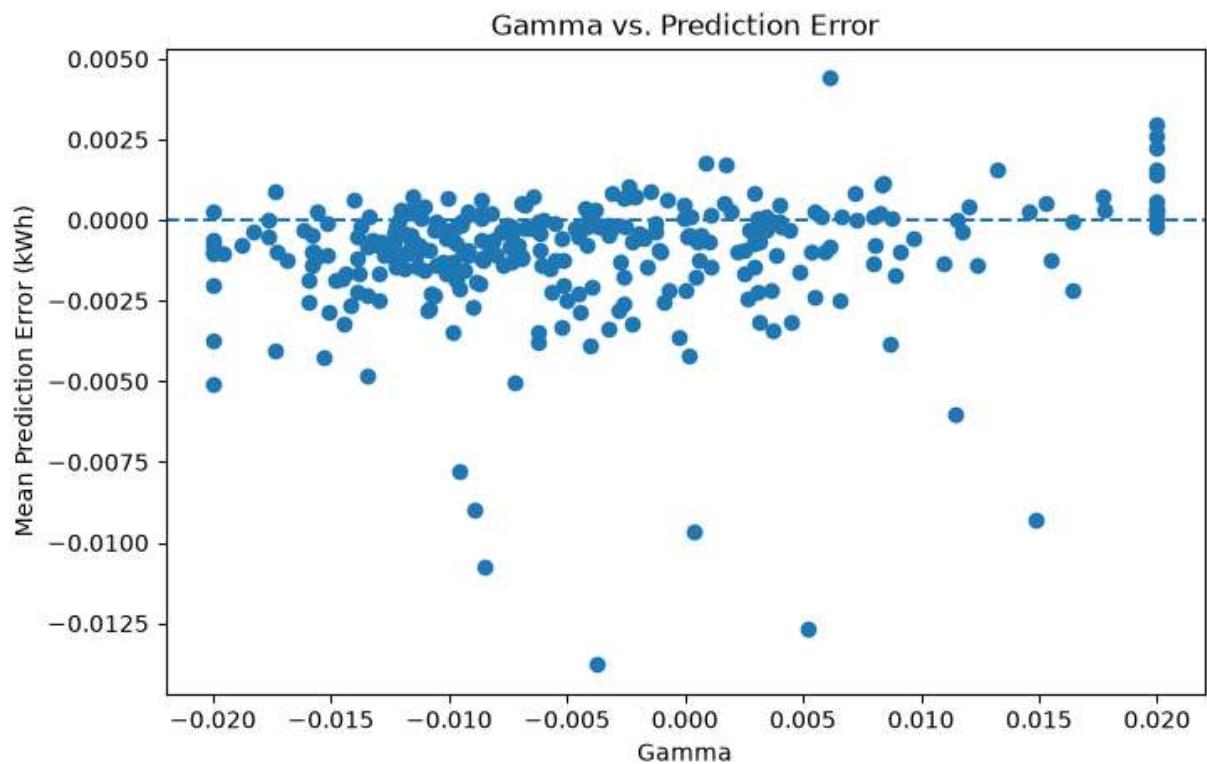
	Customer	gamma	mean_error	MAE
263	264	-0.02	-0.000782	0.033660
189	190	-0.02	-0.002014	0.054365
56	57	-0.02	-0.003752	0.089804
265	266	-0.02	0.000288	0.037688
251	252	-0.02	-0.000611	0.032194
...	...	...	...	...
62	63	0.02	0.000195	0.028077
182	183	0.02	0.000044	0.050192
214	215	0.02	0.002963	0.034575
279	280	0.02	0.002250	0.040741
31	32	0.02	0.001590	0.043246

300 rows × 4 columns

```

In [21]: plt.figure(figsize=(8, 5))
plt.scatter(error_by_customer["gamma"], error_by_customer["mean_error"])
plt.axhline(0, linestyle="--")
plt.xlabel("Gamma")
plt.ylabel("Mean Prediction Error (kWh)")
plt.title("Gamma vs. Prediction Error")
plt.show()

```



This plot suggests little relationship between γ and mean prediction error. Most errors cluster slightly below zero, so the model generally underpredicts. The few large errors are scattered. This provides evidence that γ is probably not the main source of systematic offset. Therefore the most likely problematic parameter seems to be *alpha*.

```
In [23]: alpha_check = (
    train_df.groupby("Customer")
    .agg(
        Generator_Capacity=("Generator Capacity", "first"),
        max_PPV=("PPV", "max"),
        alpha=("alpha", "first")).reset_index()

    alpha_check["max_PPV_ratio"] = (
        alpha_check["max_PPV"] / alpha_check["Generator_Capacity"])

    alpha_check.head()
```

```
Out[23]:
```

	Customer	Generator_Capacity	max_PPV	alpha	max_PPV_ratio
0	1	3.78	1.838	1.119017	0.486243
1	2	1.62	0.700	0.481045	0.432099
2	3	1.00	0.500	0.338140	0.500000
3	4	1.00	0.450	0.270474	0.450000
4	5	1.00	0.381	0.265757	0.381000

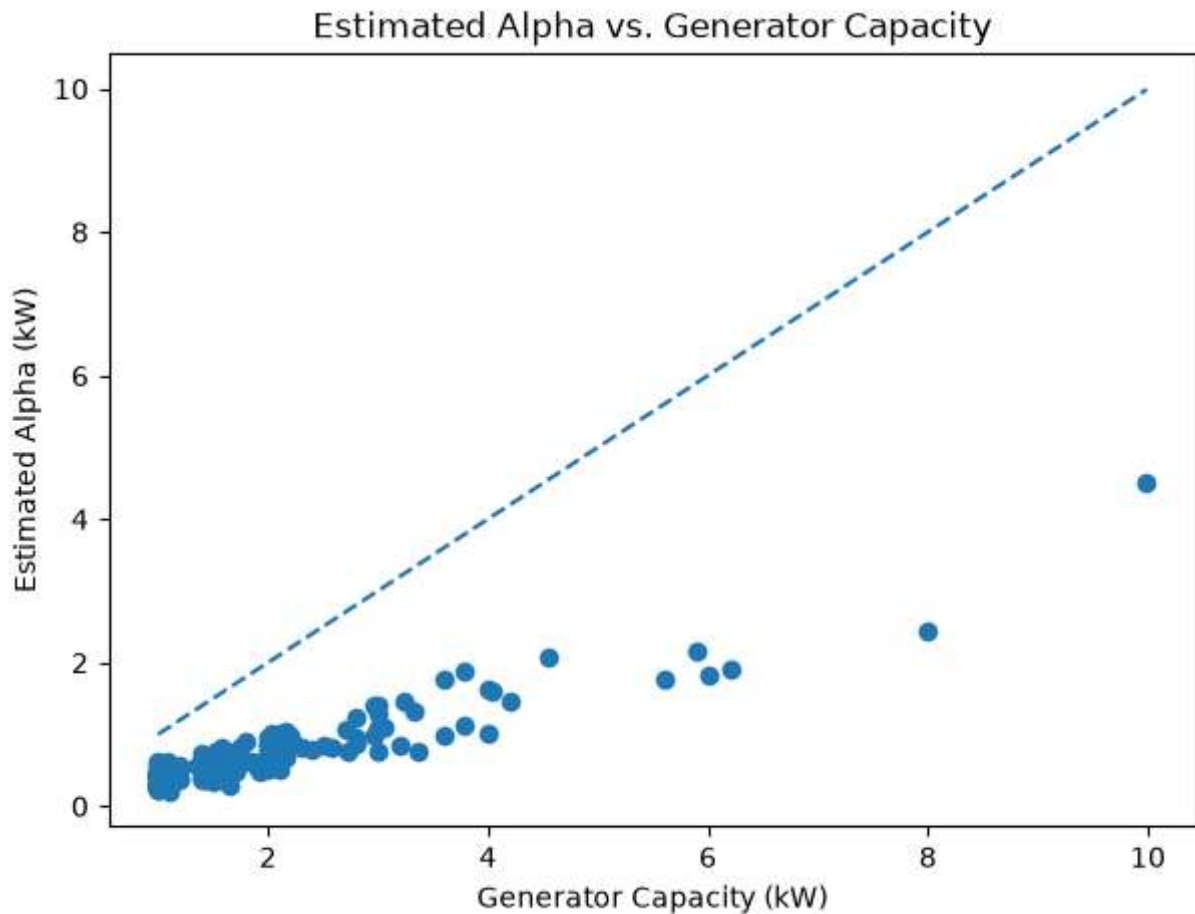
```
In [24]: plt.figure(figsize=(7, 5))

plt.scatter(
    alpha_check["Generator_Capacity"],
    alpha_check["alpha"])

x = np.linspace(
    alpha_check["Generator_Capacity"].min(),
    alpha_check["Generator_Capacity"].max(),
    100)

plt.plot(x, x, linestyle="--")

plt.xlabel("Generator Capacity (kW)")
plt.ylabel("Estimated Alpha (kW)")
plt.title("Estimated Alpha vs. Generator Capacity")
plt.show()
```



It is possible that the regression is using α to compensate for an overestimated effective irradiance term: $\frac{S(t)}{G_{\text{STC}}} \max(0, \cos(\theta_{\text{sun}} - \beta))$. In particular, our use of $S(t) \cos(\theta_{\text{sun}} - \beta)$ may be physically inconsistent because `shortwave_radiation` is already a measure of radiation received on a particular surface, while the cosine factor is intended to account for the panel's orientation relative to the incoming solar rays. We want to investigate whether `shortwave radiation` and `sun position` are redundant. First we compare the pairwise relationships and calculate their correlations.

```
In [26]: # Correlations with true PV generation
print(
    train_df[["shortwave_radiation", "sun_position", "PPV"]]
    .corr()["PPV"])
```

```
shortwave_radiation    0.422491
sun_position           -0.307815
PPV                    1.000000
Name: PPV, dtype: float64
```

```
In [27]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(
    train_df["shortwave_radiation"],
    train_df["PPV"],
    s=2)
```

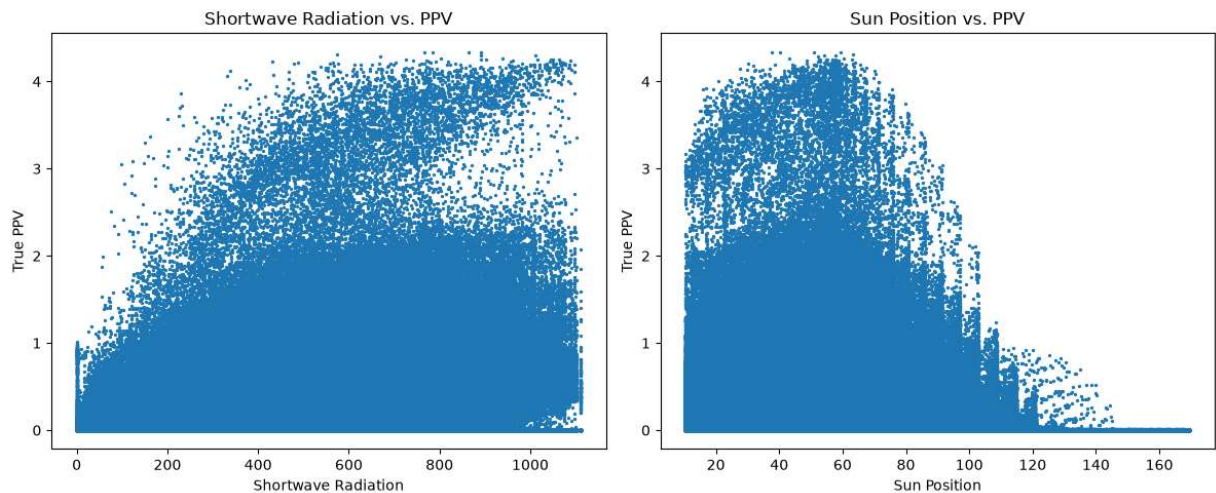
```

axes[0].set_xlabel("Shortwave Radiation")
axes[0].set_ylabel("True PPV")
axes[0].set_title("Shortwave Radiation vs. PPV")

axes[1].scatter(
    train_df["sun_position"],
    train_df["PPV"],
    s=2)
axes[1].set_xlabel("Sun Position")
axes[1].set_ylabel("True PPV")
axes[1].set_title("Sun Position vs. PPV")

plt.tight_layout()
plt.show()

```



This result shows that there is an issue with how we are using sun position. **Shortwave radiation** have a positive correlation with PPV (0.42) and shows a fairly strong increasing relationship with PV output. **Sun position** has a negative correlation with PPV (-0.31), but the scatterplot shows a nonlinear relationship. PPV is generally highest at intermediate sun positions and approaches zero at large angles. This makes sense if **sun_position** is the solar zenith angle: panels receive little direct irradiant when the sun is near the horizon. However, the fact that shortwave radiation already contains the actual measured radiation reaching the surface means it may already incorporate much of the effect of solar radiation.

Final Notes

Before finding the physics-informed regression equation, we should first investigate the predictive power of each variable independently. In my opinion, our strongest predictive variable is sun position. This is because weather data can be difficult to acquire, but if we know the location of the system, sun position can be easily calculated without needing proprietary historical weather data. In addition to sun angle, we can use some variable that can account for sunniness / cloudiness. Is there a better variable/variables from archive-api.open-meteo we could use? We should investigate the following Open-Meteo archive variables:

1. `shortwave_radiation` (GHI): Captures the effect of sun angle, clouds, aerosols, ect..
Open-Meteo defines it as global irradiation.
2. `cloud_cover` (%): Simple explicit cloudiness variable.
3. `direct_normal_irradiance` (DNI): Represents scattered radiation reaching the surface, including radiation affected by clouds. This could help explain PV production when direct sunlight is reduced.
4. `diffuse_radiation` (DHI): Represents scattered radiation reaching the surface, including radiation affected by clouds. This could help explain PV production when direct sunlight is reduced.
5. `sunlight_duration` : Measures the time during the preceding hour for which DNI exceeded the WMO sunshine threshold. This could be a useful alternative measure of cloudiness, although it is less continuous than cloud cover.